

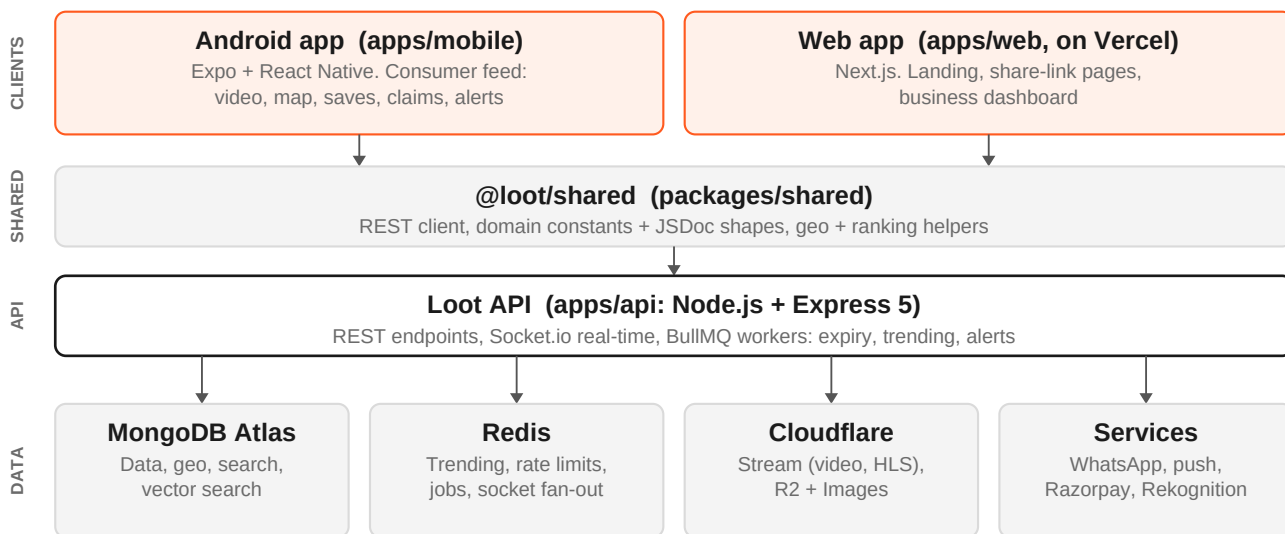
Loot Tech Stack

The stack for Loot's Android app, web app and self-written backend.

Updated October 2026.

In one paragraph

Loot uses **React Native (Expo)** for Android and **React (Next.js)** for web, both in plain **JavaScript**, sharing a package called **@loot/shared**. The backend is a **self-written API: Node.js + Express 5**, with **MongoDB Atlas** for data, location, search and recommendation vectors, **Redis** for trending, rate limits and jobs, **Socket.io** for real-time, and **Cloudflare** for video and images. Payments go through **Razorpay**. There is no Firebase.



Why this shape

- **Full control of the backend.** Every rule (auth, ranking, permissions, lifecycle) is your own code, with no platform limits and no lock-in.
- **One language end to end.** JavaScript in the API, both apps and the shared package. Plain ES modules, so Node, Next.js and Metro load the shared code as-is with no build step; zod checks data at runtime.
- **MongoDB Atlas covers several needs at once.** Location queries, text search and vector search live in one managed database instead of three.
- **The two apps do different jobs.** Android is the consumer feed; web is the landing page, share links and the business dashboard. They share domain code, not screens.

Status labels used in this document

Reuse

Already in the repo, carries over

MVP

Needed for the first release

Next

Second phase: business and money

Later

When usage grows

1. API backend

apps/api. The backend you write: one Node.js service with REST, real-time and background workers.

TOOL	WHAT IT DOES FOR LOOT	STATUS
Node 22+ and Express 5	HTTP server. Express 5 passes errors from async handlers to the error handler automatically.	Reuse
JavaScript (ESM) + ESLint	Plain Node ES modules with no build step; ESLint catches mistakes and zod checks data at runtime. Same language as the apps.	Reuse
zod	Validates every request body and query; schemas move into @loot/shared as endpoints land, so app forms use the same rules.	Reuse
Mongoose	Models, indexes and validation for MongoDB collections.	Reuse
jose (JWT)	Signs 15-minute access tokens. Refresh tokens are random, stored hashed, rotated on every use.	Reuse
WhatsApp Cloud API	Delivers login OTPs. In development the API logs the code instead of sending it.	Reuse
ioredis	Redis client: OTP rate limits today; trending sorted sets and caching next.	Reuse
BullMQ	Background jobs: expire loot at expiryAt, ending-soon alerts, trending decay every minute.	MVP
Socket.io + Redis adapter	Live counters on the loot details screen and 'new loot nearby' per geohash cell, across multiple servers.	MVP
pino + pino-http	Fast structured logs with a request id on every line.	Reuse
helmet + cors	Secure HTTP headers and an allow-list for the web app's origin.	Reuse
Push sender	Sends push notifications via the Expo push service or FCM/APNs directly — to be decided. No Firebase backend.	MVP
Cloudflare Stream + R2 SDK	Issues direct-upload URLs so phones upload media straight to Cloudflare, not through the API.	MVP
AWS Rekognition	Automatic moderation of uploaded images and video frames.	Next
Razorpay Node SDK	Orders and subscriptions for boosts and pro plans; webhooks verified by signature.	Next
Vitest + Supertest	API tests that call real routes, plus a contract test that runs the shared client against the API.	Reuse
mongodb-memory-server + ioredis-mock	Real MongoDB and fake Redis in tests, so they run with no setup.	Reuse

2. Data and infrastructure

Managed services, all in or near the Mumbai region.

TOOL	WHAT IT DOES FOR LOOT	STATUS
MongoDB Atlas (Mumbai)	Main database: users, businesses, loot, saves, claims, redemptions, reviews.	MVP
2dsphere index + \$geoNear	Nearby feed: loot within the radius, sorted by distance, then scored.	MVP
Time-series collection	Raw views, saves, claims and shares for the business analytics dashboard.	Next
Atlas Search	Search loot and businesses by name, category and tags.	Next
Atlas Vector Search	'For You' recommendations from loot and user embeddings.	Later
Redis (Upstash or Redis Cloud)	Trending scores, OTP rate limits, BullMQ queues, Socket.io fan-out.	MVP
Cloudflare Stream	Video: upload, transcoding to adaptive HLS, thumbnails, fast playback in India.	MVP
Cloudflare R2 + Images	Image storage with on-the-fly resizing and free egress.	MVP
Railway or Render	Hosts the API and workers to start; move to AWS Mumbai (ECS) at scale.	MVP

3. Android app

apps/mobile. The consumer feed, built with Expo on React Native, in JavaScript.

TOOL	WHAT IT DOES FOR LOOT	STATUS
Expo SDK + React Native	App framework: React components rendered as real Android views.	Reuse
Expo development build	Your own debug build, required for native libraries such as maps and Razorpay.	MVP
Expo Router	File-based screens and navigation; opens share links directly in the app.	MVP
EAS Build, Submit, Update	Cloud builds, Play Store upload and over-the-air JavaScript fixes.	MVP
expo-video + @shopify/flash-list	Smooth, autoplaying vertical video feed with recycled rows.	MVP
Reanimated + Gesture Handler	Swipes, double-tap to save, 60 fps animations.	MVP
expo-image + NativeWind	Cached images; Tailwind classes matching the web app.	MVP
TanStack Query + MMKV + Zustand	Server data with an on-device cache; local UI state.	MVP
expo-secure-store	Keeps the session tokens in Android's encrypted storage (the @loot/shared token store).	MVP
socket.io-client	Live counters and nearby-loot updates from the API.	MVP

TOOL	WHAT IT DOES FOR LOOT	STATUS
react-native-maps + supercluster	Map of nearby loot with clustered pins.	MVP
expo-location + expo-task-manager	Location and background geofencing for loot alerts.	MVP
expo-notifications	Push: new loot nearby, ending soon, followed business posted.	MVP
Sentry	Crash and error reports.	MVP
react-native-razorpay	In-app payments for boosts and pro plans.	Next
expo-image-picker + expo-camera	Businesses capture photos and videos for loot posts.	Next
Maestro	End-to-end tests on an emulator.	Later

4. Web app

apps/web. Landing page, share-link pages and the business dashboard, on Vercel.

TOOL	WHAT IT DOES FOR LOOT	STATUS
Next.js 16 + React 19	Server-rendered share pages that preview properly in Google and WhatsApp.	Reuse
Tailwind 4, Radix UI, lucide-react	Styling, accessible components, icons.	Reuse
TanStack Query + Zustand	Server data and local UI state.	Reuse
react-hook-form + zod	Loot creation and business onboarding forms.	Reuse
Framer Motion, qrcode.react	Landing animations; redemption QR codes.	Reuse
Vitest, Testing Library, Playwright	Unit, component and browser tests.	Reuse
REST client from @loot/shared	Data layer and phone-OTP sign-in talk to apps/api; the Firebase SDK is gone.	Reuse
Google Maps JS API	Businesses pin branch locations; maps on share pages.	MVP
next/log	Auto-generated preview images for each loot's share link.	MVP
Razorpay Checkout.js	Web payments for boosts and pro plans.	Next
Recharts + TanStack Table	Business analytics charts and tables.	Next
Sentry	Error reports from browser and server.	Next

5. Shared package and tooling

packages/shared plus how code is checked, built and shipped.

TOOL	WHAT IT DOES FOR LOOT	STATUS
@loot/shared: REST client, models, geo, ranking	configureLootClient() with token refresh, enum constants and JSDoc shapes, geo and urgency helpers.	Reuse

TOOL	WHAT IT DOES FOR LOOT	STATUS
@loot/shared: zod schemas	One set of request schemas for the API, web forms and mobile forms.	MVP
npm workspaces + Husky + lint-staged	One install for the whole repo; lint and format before each commit.	Reuse
GitHub Actions	Lint and test every pull request.	MVP
Vercel / EAS / Railway	Deploys for web, Android and the API.	MVP
PostHog	Product analytics: funnels, retention, feature flags.	Later
pnpm + Turborepo	Faster installs and cached builds once the monorepo grows.	Later

6. Design rules for the API

Decisions that are expensive to change later.

- **Loot carries its own location.** Each loot document stores a GeoJSON point copied from its branch, with a 2dsphere index. The Nearby feed is one aggregation: \$geoNear, filter active and unexpired, score, sort, cursor page.
- **Counters on documents, events in a log.** saves, claims and views are updated with \$inc; raw events go to a time-series collection for analytics.
- **Trending comes from Redis.** Engagement increments sorted sets; a job applies time decay every minute and writes trendingScore back. No single signal dominates, and boosts are capped at 1.3x.
- **Lifecycle runs on jobs.** A delayed BullMQ job fires at expiryAt, plus a periodic sweeper. Never a TTL index on loot: expired loot is archived, not deleted.
- **The database enforces uniqueness.** Unique indexes on {userId, lootId} stop double saves and claims.
- **Auth is hardened from day one.** OTPs and refresh tokens are stored hashed, OTP requests are rate-limited per phone and IP, and reusing an old refresh token revokes the whole session.
- **One error shape.** Every error response is { error: { code, message, details } }; clients switch on the stable code.
- **Only professional accounts create loot,** enforced by middleware on every write route.

7. Build order

Tracked in .claude/tasks/backend-rewrite/.

- **Phase 0: foundations — done.** apps/api scaffold, config, logging, MongoDB and Redis, OTP login with JWT and refresh tokens, tests.
- **Phase 1: core API — next.** Profiles, businesses and branches, loot, media upload, the four feeds, save and claim, lifecycle jobs, trending, push, real-time.
- **Phase 2: clients — partly done.** The REST client in @loot/shared and the web app off Firebase are done; remaining: shared zod schemas and the Android MVP.
- **Phase 3: business and money.** Razorpay, analytics, moderation and reports.
- **Phase 4: launch.** CI and deploys. The Firebase backend is already retired and its old data is not migrated.

8. Considered and not chosen

Alternatives weighed while picking this stack.

- **Firebase.** Fast to start, but per-read pricing hurts a scrolling feed and ranking is split across services.
- **Supabase (Postgres).** Less code to write, but chosen against in favour of owning the whole backend.
- **TypeScript.** Dropped for now to keep the codebase simple; JSDoc shapes and zod validation cover the critical contracts. Can be brought back later.
- **Flutter or native Kotlin.** Flutter's web output hurts share-link SEO; Kotlin means a second language.
- **Plain React (Vite) for web.** Browser-only rendering breaks share-link previews; Next.js renders on the server.
- **Redux, GraphQL.** TanStack Query + Zustand and a REST client cover the same needs with less code.